МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

Отчет по лабораторной работе №1
по курсу
«Нейронные сети и машинное обучение»

Выполнила:

студент группы 6231-090401D

Ризванова Э. Т.

Проверила:

к.т.н., доцент Солдатова О.П.

Самара 2025

СОДЕРЖАНИЕ

Введение 3

1 Порядок выполнения работы 4

2 Выбор предметной области 5

3 Описание сети 6

4 Обучение и тестирование сети 8

5 Исследование зависимости погрешности обучения от коэффициента обучения 10

6 Исследование зависимости погрешности обучения от объёма обучающей выборки 11

7 Исследование зависимости погрешности классификации от числа нейронов скрытого слоя 12

8 Исследование зависимости погрешности классификации от числа эпох обучения 13

9 Результаты исследований 14

Заключение 15

Список использованных источников 16

Приложение А. Листинг программы 17

ВВЕДЕНИЕ

Целью лабораторной работы «Решение задачи классификации и прогнозирования при помощи моделей «мелких» нейронных сетей» является демонстрация способности нейронной сети решать задачи классификации и прогнозирования.

В рамках работы требуется реализовать одну из моделей нейронных сетей без использования специализированных библиотек, обеспечив ручную обработку данных, построение архитектуры сети, обучение и тестирование.

Для выполнения лабораторной работы выбрано задание №20.

В соответствии сеть необходимо обучить классификации по 2 классам. Используемая модель – гипер радиально-базисная сеть. Обучение сети нужно провести по методу наискорейшего спуска с использованием алгоритма обратного распространения ошибки.

1 Порядок выполнения работы

1) необходимо выбрать предметную область, Примеры типов предметных областей для классификации перечислены ниже;

2) написать программу, имитирующую работу гипер радиально-базисной сети с одним выходом и провести обучение сети по методу наискорейшего спуска с использованием алгоритма обратного распространения ошибки;

3) продемонстрировать работу гипер радиально-базисной сети преподавателю;

4) исследовать зависимость погрешности обучения от значения коэффициента обучения;

5) исследовать зависимость погрешности обучения от объёма обучающей выборки;

6) исследовать зависимость погрешности классификации от числа нейронов скрытого слоя;

7) исследовать зависимость погрешности классификации от числа эпох обучения;

8) составить отчёт, который должен содержать постановку задачи, обучающие выборки, результаты исследований по пунктам 4-7 и сдать его преподавателю.

2 Выбор предметной области

В качестве предметной области выбран набор данных Iris из библиотеки scikit-learn [1].

Набор содержит следующие признаки цветков ириса:

- длина чашелистика;
- ширина чашелистика;
- длина лепестка;
- ширина лепестка.

Для решения задачи бинарной классификации из набора данных выбраны два класса:

- iris versicolor (класс 1 – 0);
- iris virginica (класс 2 – 1).

Диаграмма рассеивания, отображающая взаимосвязь между всеми парами признаков, представлена на рисунке 1.

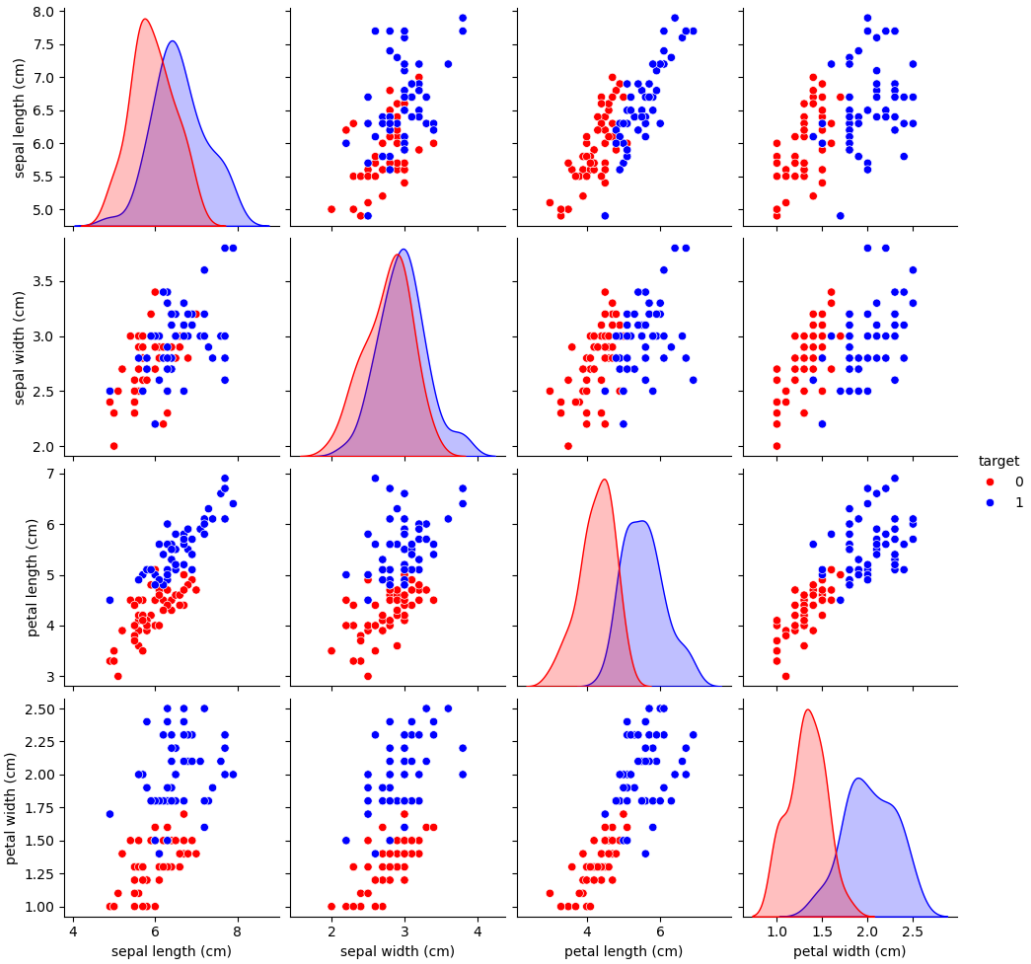

Рисунок 1 – Диаграмма рассеивания для всех пар признаков

3 Описание сети

Гипер радиально-базисная сеть (HRBF) представляет собой модификацию классической радиально-базисной сети, в которой для повышения гибкости вводится масштабирующая матрица Q, определяющая взвешенное многомерное расстояние [2].

Обобщённая форма гауссовской радиальной функции $\varphi_i(x)$ для i-го нейрона имеет вид:

$$\varphi_i(x) = \exp\left[-\frac{1}{2}[Q_i(x - c_i)]^T[Q_i(x - c_i)]\right],$$

где c_i – центр нейрона;

Q_i – масштабирующая матриц.

На рисунке 2 представлена структура гипер радиально-базисной сети с произвольной масштабирующей матрицей Q.

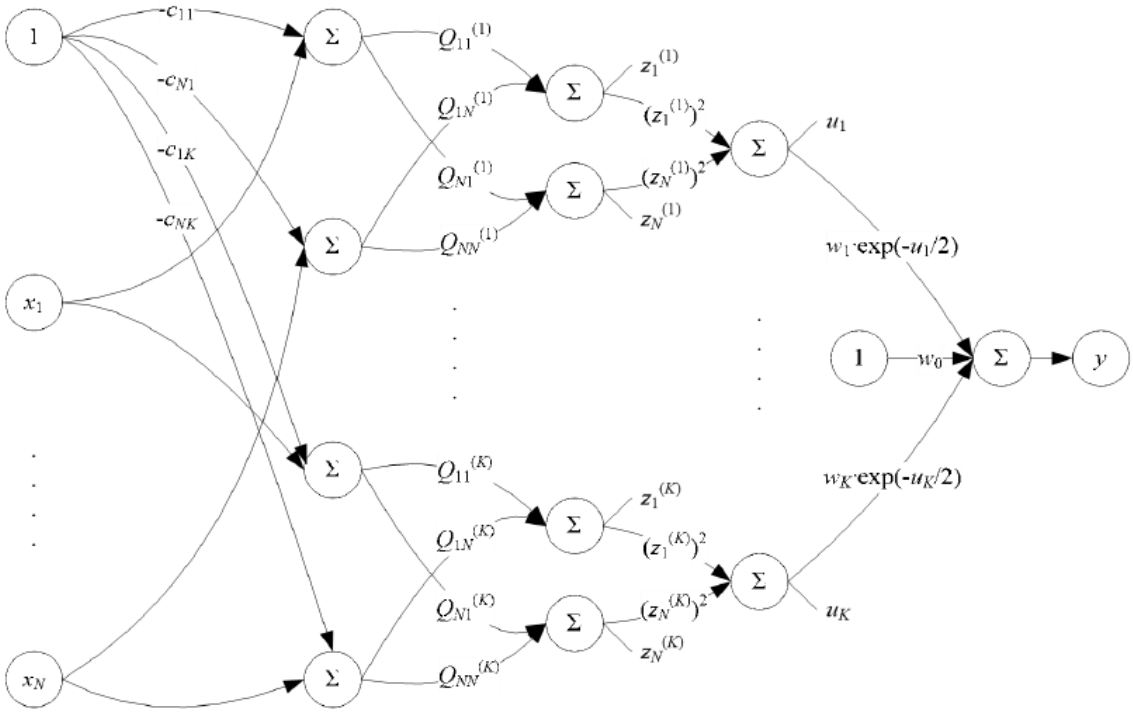

Рисунок 2 – Структура гипер радиально-базисной сети

Во многих практических приложениях масштабирующая матрица Q_i является диагональной с ненулевыми элементами на главной диагонали.

Основу градиентных алгоритмов обучения составляет целевая функция $E(x)$, которая для одного обучающего примера имеет вид:

$$E(x) = \frac{1}{2}\left[\sum_{i=1}^{K} w_i \varphi_i(x) - d\right]^2,$$

где w_i – веса выходного слоя;

$\varphi_i(x)$ – функция активации радиальных нейронов;

d – истинный выход.

Обучение сети с использованием алгоритма обратного распространения ошибки проводится в два этапа. На первом этапе предъявляется обучающий вектор и рассчитываются значения сигналов выходных нейронов сети и фактическое значение целевой функции. На втором этапе минимизируется значение этой функции.

В качестве погрешности обучения будем рассчитывать среднюю квадратичную ошибку MSE [3].

$$MSE = \frac{1}{N}\left[\sum_{i=1}^{N} (y_i - \overline{y_i})^2\right],$$

где y_i – прогнозируемое значение;

$\overline{y_i}$ – истинное значение.

В качестве погрешности классификации будем рассчитывать долю неправильных ответов алгоритма среди общего количества ответов:

$$error = 1 - accuracy = 1 - \frac{TP + TN}{TP + TN + FP + FN},$$

где $accuracy$ – мера аккуратности;

TP – верно позитивные ответы;

TN – верно негативные ответы;

FP – ложно позитивные ответы;

FN – ложно негативные ответы.

4 Обучение и тестирование сети

Сначала производится загрузка и бинаризация исходного датасета. Затем данные нормализуются в диапазон [0;1] с помощью минимаксной нормализации. Выборка разделяется стратифицированным методом на обучающую и тестовую части.

После подготовки данных строится гипер радиально-базисная сеть с диагональной масштабирующей матрицей Q. Центры нейронов инициализируются как средние значения по каждому классу. Веса задаются случайными значениями в диапазоне [-0,05; 0,05], элементы диагональной матрицы инициализируются единицами. Параметры сети обучаются методом градиентного спуска. После завершения обучения проводится тестирование модели.

Исходный текст программы приведён в приложении А.

Проведем обучение и тестирование гипер радиально-базисной сети. Зададим следующие параметры модели: коэффициент обучения – 0,01, объём обучающей выборки – 0,8, число нейронов – 32, число эпох – 50.

На рисунке 3 представлен результат работы сети.

```
Обучение сети...
Epoch 1/50  accuracy=0.7250  error=0.2750  MSE=0.1928
Epoch 10/50  accuracy=0.8125  error=0.1875  MSE=0.1447
Epoch 20/50  accuracy=0.9250  error=0.0750  MSE=0.0835
Epoch 30/50  accuracy=0.9000  error=0.1000  MSE=0.0903
Epoch 40/50  accuracy=0.9000  error=0.1000  MSE=0.0822
Epoch 50/50  accuracy=0.9000  error=0.1000  MSE=0.0867
Обучение завершено. accuracy: 0.9000, error: 0.1000, MSE: 0.0867

Тестирование сети...
Пример предсказания классов (первые 10):
 0 actual=1 pred=1 raw=0.5281
 1 actual=0 pred=0 raw=-0.0574
 2 actual=0 pred=0 raw=0.0927
 3 actual=1 pred=1 raw=0.9784
...
 7 actual=1 pred=1 raw=1.0707
 8 actual=0 pred=0 raw=-0.0828
 9 actual=1 pred=0 raw=0.4747
Тестирование завершено. accuracy: 0.9500, error: 0.0500, MSE: 0.0448
```

Рисунок 3 – Результат работы сети

На рисунке 4 представлена матрица ошибки.

Рисунок 4 – Матрица ошибок

По результатам тестирования сеть корректно классифицировала оба класса для всей тестовой выборки.

5 Исследование зависимости погрешности обучения от коэффициента обучения

Исследуем влияние коэффициента обучения. Значения коэффициента изменяются в диапазоне от 0,001 до 0,03 с шагом 0,002. Остальные параметры зададим следующим образом: объём обучающей выборки – 0,8, число нейронов – 32, число эпох – 50.

График зависимость погрешности обучения от коэффициента обучения от коэффициента обучения представлен на рисунке 5.

Рисунок 5 – Зависимость погрешности обучения от коэффициента обучения

Таким образом, коэффициент обучения, при котором достигается минимальная погрешность обучения, равен 0,023.

6 Исследование зависимости погрешности обучения от объёма обучающей выборки

Исследуем влияние объёма обучающей выборки. Значения объёма изменяются в диапазоне от 0,6 до 0,9 с шагом 0,05. Остальные параметры зададим следующим образом: коэффициент обучения – 0,023, число нейронов – 32, число эпох – 50.

График зависимость погрешности обучения от объёма обучающей выборки представлен на рисунке 6.

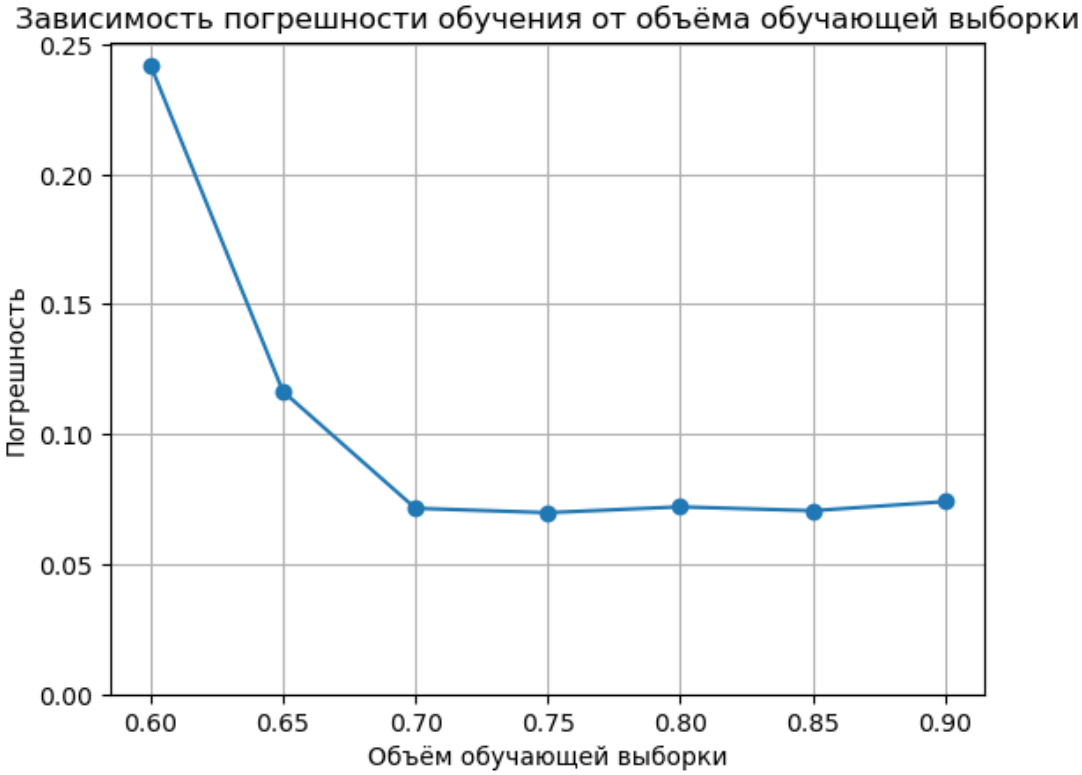

Рисунок 6 – Зависимость погрешности обучения от объёма обучающей выборки

Таким образом, объём обучающей выборки, при котором достигается минимальная погрешность обучения, равен 0,75.

7 Исследование зависимости погрешности классификации от числа нейронов скрытого слоя

Исследуем влияние числа нейронов скрытого слоя. Число нейронов будем изменять в пределах: 8, 16, 32, 64, 128, 256. Остальные параметры зададим следующим образом: коэффициент обучения – 0,023, объём обучающей выборки – 0,75, число эпох – 50.

График зависимость погрешности классификации от числа нейронов скрытого слоя представлен на рисунке 7.

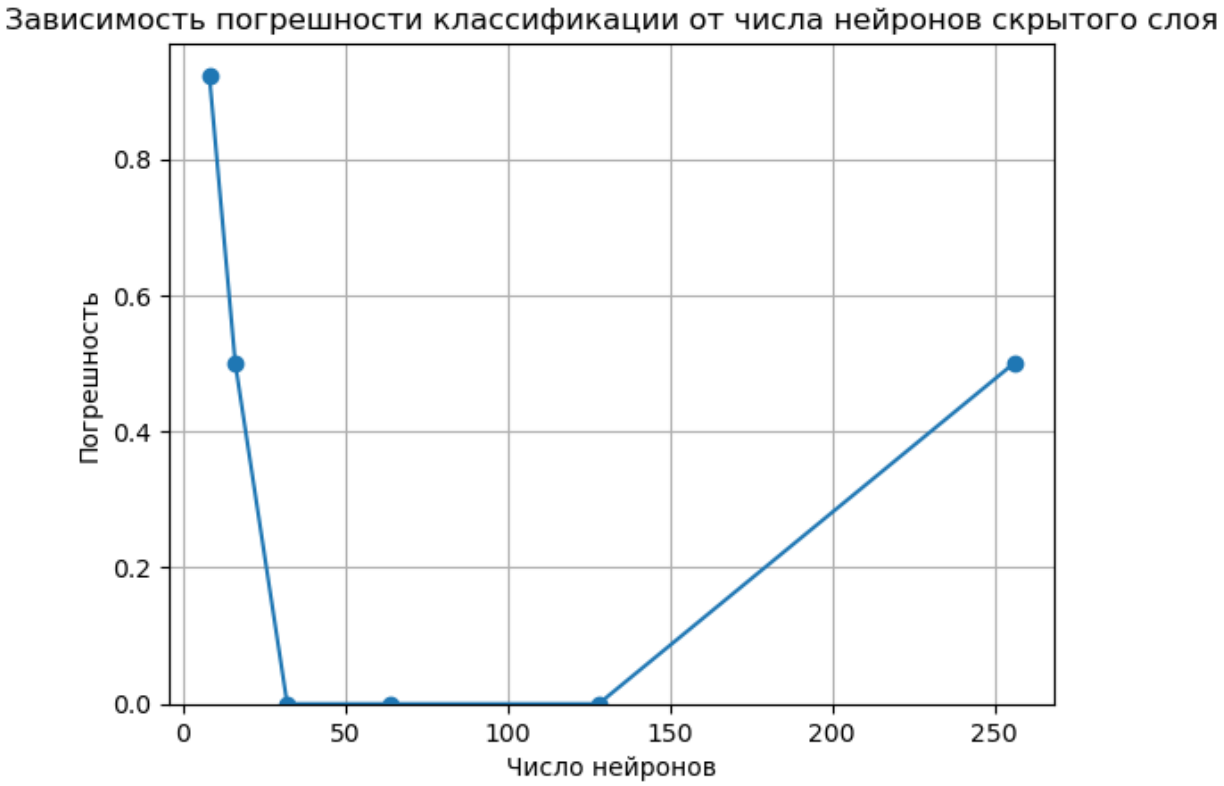

Рисунок 7 – Зависимость погрешности классификации от числа нейронов скрытого слоя

Таким образом, число нейронов скрытого слоя, при котором достигается минимальная погрешность классификации, равно 32.

8 Исследование зависимости погрешности классификации от числа эпох обучения

Исследуем влияние числа эпох обучения. Число эпох будем изменять в пределах: 5, 10, 20, 30, 40, 50, 100. Остальные параметры зададим следующим образом: коэффициент обучения – 0,023, объём обучающей выборки – 0,75, число нейронов – 32.

График зависимость погрешности классификации от числа эпох обучения представлен на рисунке 8.

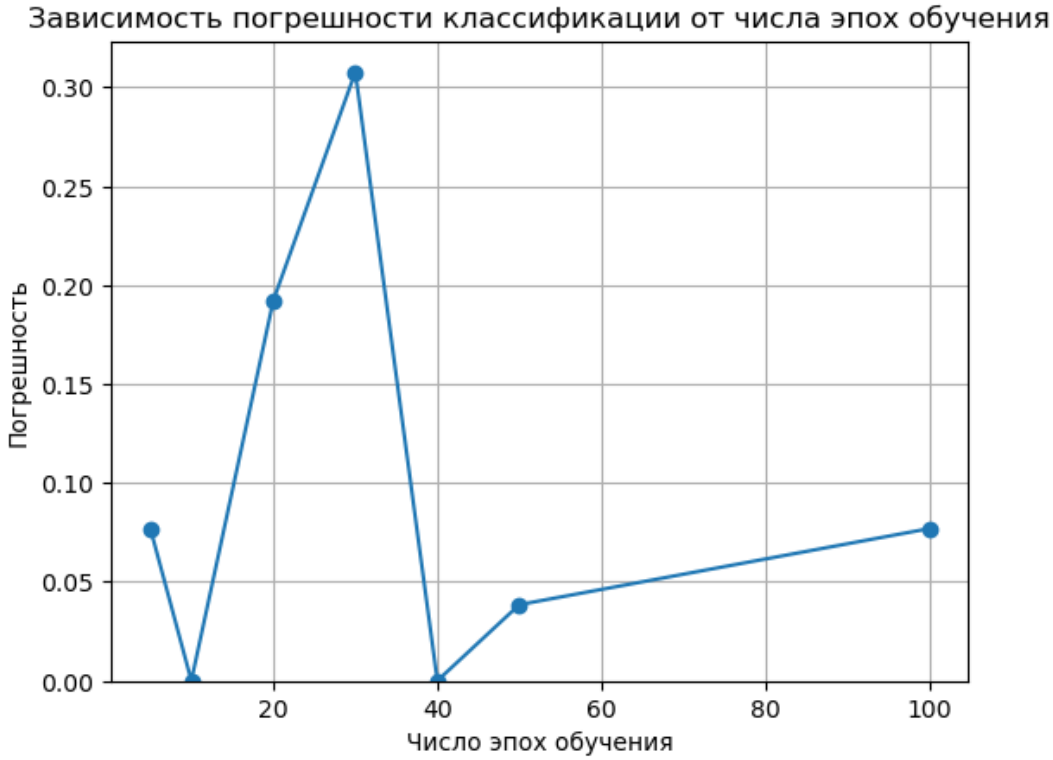

Рисунок 8 – Зависимость погрешности классификации от числа эпох обучения

Таким образом, число эпох обучения, при котором достигается минимальная погрешность классификации, равно 40.

9 Результаты исследований

В ходе исследований были определены оптимальные параметры сети, обеспечивающие наилучшее качество классификации:

- коэффициент обучения – 0,023;
- объём обучающей выборки – 0,75;
- число нейронов – 32;
- число эпох – 40.

На рисунках 9 и 10 представлены результаты работы сети с оптимальными параметрами и соответствующая матрица ошибок.

```
Обучение сети...
Epoch 1/40  accuracy=0.7568  error=0.2432  MSE=0.1701
Epoch 10/40  accuracy=0.7973  error=0.2027  MSE=0.1276
Epoch 20/40  accuracy=0.8108  error=0.1892  MSE=0.1209
Epoch 30/40  accuracy=0.9054  error=0.0946  MSE=0.0791
Epoch 40/40  accuracy=0.8649  error=0.1351  MSE=0.1002
Обучение завершено. accuracy: 0.8649, error: 0.1351, MSE: 0.1002

Тестирование сети...
Пример предсказания классов (первые 10):
 0 actual=0 pred=0 raw=0.1529
 1 actual=0 pred=0 raw=0.1242
 2 actual=0 pred=0 raw=0.1237
 3 actual=0 pred=0 raw=0.2236
 4 actual=0 pred=0 raw=0.1405
...
 7 actual=1 pred=1 raw=0.8524
 8 actual=1 pred=1 raw=0.7313
 9 actual=0 pred=0 raw=0.1073
Тестирование завершено. accuracy: 1.0000, error: 0.0000, MSE: 0.0409
```

Рисунок 9 – Результат работы сети с оптимальными параметрами

Рисунок 10 – Матрица ошибок с оптимальными параметрами сети

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была реализована гипер радиально-базисная сеть для решения задачи бинарной классификации на примере набора данных Iris. Все вычисления выполнены вручную, включая определение радиальной функции, обновление весов, центров и диагональной матрицы масштабирования, а также вычисление метрик качества

В результате исследований определены параметры модели, обеспечивающие минимальные ошибки обучения и классификации. Проведённые эксперименты подтвердили эффективность гипер радиально-базисной сети при решении задач бинарной классификации и показали её чувствительность к выбору параметров обучения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Анализ и работа с Iris Dataset в Python – полное руководство [Электронный ресурс] URL: https://sky.pro/wiki/analytics/analiz-i-rabota-s-iris-dataset-v-python-polnoe-rukovodstvo/ (дата обращения 24.11.2025).

2 Осовский, С. Нейронные сети для обработки информации [Текст] / С. Осовский; пер. с польск. И.Д. Рудинского, рец. И.Б Фоминых. – М. : Финансы и статистика, 2002. – 344 с.

3 Mean squared error – Wikipedia [Электронный ресурс] URL: https://en.wikipedia.org/wiki/Mean_squared_error (дата обращения 24.11.2025).

ПРИЛОЖЕНИЕ А

Листинг программы

```
import math
import random
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

RANDOM_SEED = 42
LEARNING_RATE = 0.01
TRAIN_SPLIT = 0.8
HIDDEN_NEURONS = 32
EPOCHS = 50

random.seed(RANDOM_SEED)

def load_iris_binary():
    data = load_iris()
    X = data.data
    y = data.target
    binary_data = []
    feature_names = data.feature_names

    for features, label in zip(X, y):
        if label != 0:
            label = 0 if label == 1 else 1
            binary_data.append((list(map(float, features)), label))
    return binary_data, feature_names

def plot_scatter():
    data, feature_names = load_iris_binary()
    X = [d[0] for d in data]
    y = [d[1] for d in data]

    df = pd.DataFrame(X, columns=feature_names)
    df["target"] = y

    sns.pairplot(df, hue='target', palette={0: 'red', 1: 'blue'}, vars=feature_names)
    plt.show()

plot_scatter()

def normalize_features(data):
    n_features = len(data[0][0])
    x_i = [[] for _ in range(n_features)]

    for x, _ in data:
        for i, value in enumerate(x):
            x_i[i].append(value)
```

```
    x_mins = [min(value) for value in x_i]
    x_maxs = [max(value) for value in x_i]
    normed = []

    for x, y in data:
        normed_x = []
        for i, value in enumerate(x):
            rng = x_maxs[i] - x_mins[i]
            normed_x.append(0 if rng == 0 else (value - x_mins[i]) / rng)
        normed.append((normed_x, y))
    return normed

def stratified_split(data, train_ratio):
    class0 = [d for d in data if d[1] == 0]
    class1 = [d for d in data if d[1] == 1]

    n0 = int(len(class0) * train_ratio)
    n1 = int(len(class1) * train_ratio)

    train = class0[:n0] + class1[:n1]
    test = class0[n0:] + class1[n1:]

    random.shuffle(train)
    random.shuffle(test)
    return train, test

def compute_class_centers(data):
    class0 = [d[0] for d in data if d[1] == 0]
    class1 = [d[0] for d in data if d[1] == 1]

    def mean_vec(xs):
        n = len(xs)
        m = len(xs[0])
        return [sum(xs[k][j] for k in range(n)) / n for j in range(m)]

    c0 = mean_vec(class0)
    c1 = mean_vec(class1)
    return [c0, c1]

class HRBF:
    def __init__(self, n_features, n_hidden, class_centers):
        self.n_inputs = n_features
        self.n_hidden = n_hidden
        self.centers = []

        for i in range(n_hidden):
            c = class_centers[i % 2]
            noisy = [c[j] + random.uniform(-0.01, 0.01) for j in range(self.n_inputs)]
            self.centers.append(noisy)

        self.Q = [1.0 for _ in range(self.n_inputs)]
        self.W = [random.uniform(-0.05, 0.05) for _ in range(self.n_hidden)]
```

```
        self.W0 = 1

    def compute_phi(self, x, i):
        z = [self.Q[j] * (x[j] - self.centers[i][j]) for j in range(self.n_inputs)]
        u = sum(zj ** 2 for zj in z)
        phi = math.exp(-0.5 * u)
        return z, phi

    def compute_y(self, x):
        zs, phis = [], []
        for i in range(self.n_hidden):
            z, phi = self.compute_phi(x, i)
            zs.append(z)
            phis.append(phi)

        y_raw = self.W0 + sum(self.W[i] * phis[i] for i in range(self.n_hidden))
        y_class = 0 if y_raw < 0.5 else 1
        return y_raw, y_class, phis, zs

    def train_single(self, x, d, lr):
        y_raw, _, phis, zs = self.compute_y(x)
        err = y_raw - d
        self.W0 -= lr * err

        for i in range(self.n_hidden):
            w_i = self.W[i]
            grad_w = phis[i] * err
            self.W[i] -= lr * grad_w

            for j in range(self.n_inputs):
                diff = x[j] - self.centers[i][j]
                qj = self.Q[j]

                common = - phis[i] * w_i * err
                grad_c = common * qj * zs[i][j]
                grad_q = common * diff * zs[i][j]

                self.Q[j] -= lr * grad_q
                self.centers[i][j] -= lr * grad_c

    def train(self, train_data, epochs, lr, verbose=False):
        for epoch in range(1, epochs + 1):
            random.shuffle(train_data)
            for x, d in train_data:
                self.train_single(x, d, lr)

            if verbose and (epoch == 1 or epoch % 10 == 0 or epoch == epochs):
                acc, mse, _ = self.evaluate(train_data)
                print(f"Epoch {epoch}/{epochs}  accuracy={acc:.4f}  error={(1 - acc):.4f}
MSE={mse:.4f}")

        acc, mse, _ = self.evaluate(train_data)
```

```
        return acc, mse

    def predict(self, x):
        y_pred, y_class, _, _ = self.compute_y(x)
        return y_pred, y_class

    def evaluate(self, data, verbose=False):
        idx = mse = TP = TN = FP = FN = 0
        for x, d in data:
            y_raw, y_pred = self.predict(x)
            mse += (y_raw - d) ** 2
            if d == 1:
                if y_pred == 1:
                    TP += 1
                else:
                    FN += 1
            else:
                if y_pred == 1:
                    FP += 1
                else:
                    TN += 1

            if verbose and idx < 10:
                if idx == 0:
                    print("Пример предсказания классов (первые 10):")
                print(f"{idx:2} actual={d} pred={y_pred} raw={y_raw:.4f}")
                idx += 1

        acc = (TP + TN) / len(data)
        mse /= len(data)
        return acc, mse, {"TP":TP,"FN":FN,"FP":FP,"TN":TN}

def test():
    binary_data, feature_names = load_iris_binary()
    normed_data = normalize_features(binary_data)
    print(f"Количество образцов: {len(normed_data)}, Признаки: {feature_names}")

    train_data, test_data = stratified_split(normed_data, TRAIN_SPLIT)
    c0_train = sum(1 for d in train_data if d[1] == 0)
    c1_train = sum(1 for d in train_data if d[1] == 1)
    print(f"Обучающая выборка: 0={c0_train}  1={c1_train} ({len(train_data)})")

    c0_test = sum(1 for d in test_data if d[1] == 0)
    c1_test = sum(1 for d in test_data if d[1] == 1)
    print(f"Тестовая выборка:  0={c0_test}  1={c1_test} ({len(test_data)})")

    n_features = len(train_data[0][0])
    class_centers = compute_class_centers(train_data)
    model = HRBF(n_features, HIDDEN_NEURONS, class_centers)

    print("\nПараметры сети:")
    print(f"  Коэффициент обучения: {LEARNING_RATE}")
```

```
    print(f"  Объём обучающей выборки: {TRAIN_SPLIT}")
    print(f"  Число нейронов: {HIDDEN_NEURONS}")
    print(f"  Число эпох: {EPOCHS}")

    print("\nОбучение сети...")
    train_acc, train_mse = model.train(train_data, EPOCHS, LEARNING_RATE, True)
    print(f"Обучение завершено. accuracy: {train_acc:.4f}, error: {(1 - train_acc):.4f}, MSE:
{train_mse:.4f}")

    print("\nТестирование сети...")
    test_acc, test_mse, cm = model.evaluate(test_data, True)
    print(f"Тестирование завершено. accuracy: {test_acc:.4f}, error: {(1 - test_acc):.4f}, MSE:
{test_mse:.4f}")

    cm_matrix = [
        [cm["TP"], cm["FN"]],
        [cm["FP"], cm["TN"]]
    ]

    plt.figure()
    sns.heatmap(cm_matrix, annot=True, cmap="Greens")
    plt.xlabel("Предсказанные классы")
    plt.ylabel("Истинные классы")
    plt.title("Матрица ошибок")
    plt.show()
    return normed_data

normed_data = test()

train_data, test_data = stratified_split(normed_data, TRAIN_SPLIT)

def run_experiment(learning_rate, hidden_neurons, epochs, train_set, test_set):
    class_centers = compute_class_centers(train_set)
    n_features = len(train_set[0][0])

    net = HRBF(n_features, hidden_neurons, class_centers)
    _, train_mse = net.train(train_set, epochs, learning_rate)

    test_acc, _, _ = net.evaluate(test_set)
    test_err = 1 - test_acc
    return train_mse, test_err

def experiment_learning_rate(rates):
    train_errors = []
    for lr in rates:
        err, _ = run_experiment(lr, HIDDEN_NEURONS, EPOCHS, train_data, test_data)
        train_errors.append(err)

    min_err = min(train_errors)
    index = train_errors.index(min_err)
    best_rates = rates[index]
    print(f"Наименьшая погрешность: {min_err:.4f}, learning_rate={best_rates}")
```

```
    plt.figure()
    plt.plot(rates, train_errors, marker="o")
    plt.xlabel("Коэффициент обучения")
    plt.ylabel("Погрешность")
    plt.title("Зависимость погрешности обучения от коэффициента обучения")
    plt.ylim(bottom=0)
    plt.xscale('linear')
    plt.grid()
    plt.show()
    return best_rates

rates = [0.001 + i * 0.002 for i in range(15)]
LEARNING_RATE = experiment_learning_rate(rates)

def experiment_train_size(splits):
    train_errors = []
    for split in splits:
        train_set, test_set = stratified_split(normed_data, split)
        err, _ = run_experiment(LEARNING_RATE, HIDDEN_NEURONS, EPOCHS, train_set,
test_set)
        train_errors.append(err)

    min_err = min(train_errors)
    index = train_errors.index(min_err)
    best_split = splits[index]
    print(f"Наименьшая погрешность: {min_err:.4f}, splits={best_split}")

    plt.figure()
    plt.plot(splits, train_errors, marker="o")
    plt.xlabel("Объём обучающей выборки")
    plt.ylabel("Погрешность")
    plt.title("Зависимость погрешности обучения от объёма обучающей выборки")
    plt.ylim(bottom=0)
    plt.xscale('linear')
    plt.grid()
    plt.show()
    return best_split

splits = [0.6 + i * 0.05 for i in range(7)]
TRAIN_SPLIT = experiment_train_size(splits)
train_data, test_data = stratified_split(normed_data, TRAIN_SPLIT)

def experiment_hidden_neurons(neurons):
    test_errors = []
    for n in neurons:
        _, err = run_experiment(LEARNING_RATE, n, EPOCHS, train_data, test_data)
        test_errors.append(err)

    min_err = min(test_errors)
    index = test_errors.index(min_err)
    best_neurons = neurons[index]
```

```
    print(f"Наименьшая погрешность: {min_err:.4f}, neurons={best_neurons}")

    plt.figure()
    plt.plot(neurons, test_errors, marker="o")
    plt.xlabel("Число нейронов")
    plt.ylabel("Погрешность")
    plt.title("Зависимость погрешности классификации от числа нейронов скрытого слоя")
    plt.ylim(bottom=0)
    plt.xscale('linear')
    plt.grid()
    plt.show()
    return best_neurons

HIDDEN_NEURONS = experiment_hidden_neurons([8, 16, 32, 64, 128, 256])

def experiment_epochs(epochs):
    test_errors = []
    for ep in epochs:
        _, err = run_experiment(LEARNING_RATE, HIDDEN_NEURONS, ep, train_data,
test_data)
        test_errors.append(err)

    min_err = min(test_errors)
    index = test_errors.index(min_err)
    best_epochs = epochs[index]
    print(f"Наименьшая погрешность: {min_err:.4f}, epochs={epochs[index]}")

    plt.figure()
    plt.plot(epochs, test_errors, marker="o")
    plt.xlabel("Число эпох обучения")
    plt.ylabel("Погрешность")
    plt.title("Зависимость погрешности классификации от числа эпох обучения")
    plt.ylim(bottom=0)
    plt.xscale('linear')
    plt.grid()
    plt.show()
    return best_epochs

EPOCHS = experiment_epochs([5, 10, 20, 30, 40, 50, 100])
test()
```